

Mil disculpas por el desvío de rumbo; me fui por la tangente asumiendo un roadmap de P&ID tradicional cuando el plan maestro real es una **estrategia impecable de reingeniería de datos e ingeniería de software**.

Viendo la estructura de tu esquema, el flujo es perfecto y súper profesional:



Roadmap Maestro del Proyecto (Alineado al 100%)

FASE 1: Aprender ISA ✓ (Completado)

FASE 2: Crear Diccionario ISA ✓ (Completado)

FASE 3: Inventario PLC ⌚ (¡Arrancamos acá!)

FASE 4: Convención PLC + Sistema de Gestión de Tags con Python y Base de Datos Local

FASE 5: Renombrar

FASE 6: Manual



FASE 3: Inventario PLC

En esta **Fase 3**, el objetivo principal es relevar, estructurar y catalogar **todos los tags y variables que existen actualmente en los PLC del Ingenio La Florida** (sin modificarlos todavía), para generar la línea base sobre la cual trabajaremos.

Para llevar a cabo el **Inventario del PLC**, propongo estructurar el trabajo en los siguientes pilares de levantamiento de datos:



Estructura Propuesta para el Inventario

Extracción y Formato del Archivo Fuente:

Exportación de la base de datos de tags desde la plataforma de control (ej. exportación a `.L5X` , `.CSV` o `.TXT` desde Studio 5000 / RSLogix 5000).

Campos Clave a Mapear por Tag:

Tag Actual (Raw Tag): Nombre exacto como figura hoy en el PLC.

Descripción Original: Texto descriptivo actual (si existe).

Tipo de Dato (Data Type): `BOOL` , `REAL` , `INT` , `DINT` , o tipos definidos por el usuario (`UDT`).

Scope (Alcance): Controller Tags vs. Program Tags.

Direccionamiento I/O Físico: Módulo de E/S asociado (`Rack/Slot/Channel` o dirección de red).

Área / Proceso Asociado: Molienda, Clarificación, Evaporación, Cocimiento, Calderas, etc.

Estado de Calidad de Datos: Identificación de tags huérfanos, duplicados, no utilizados (Unused Tags) o descripciones incompletas.

💡 ¿Cómo querés que procedamos?

Podemos abordarlo de dos formas según la información que tengas disponible:

Si tenés un archivo exportado (.CSV , .L5X o lista en Excel): Me podés compartir una muestra o la estructura de columnas para que armemos el análisis y la limpieza de datos.

Si querés que definamos la plantilla/estándar de inventario: Diseñamos el modelo exacto en tabla/Excel que vas a usar para volcar los tags extraídos del PLC antes de pasar a la Fase 4 (Python + BD).

¿Cómo tenés configurados o exportados los tags del PLC actualmente?

bien. definamos los sprints de la fase 3 primero. yo hago la notacion desde el sprint0

¡Excelente idea! Arrancar desde un **Sprint 0** es la mejor práctica en ingeniería de software y datos para sentar las bases, definir los entregables y estructurar la metodología antes de ejecutar las tareas.

Estructuremos los **Sprints de la Fase 3: Inventario PLC** con objetivos súper claros, delimitados y orientados a generar la base de datos limpia que utilizaremos en la Fase 4.



FASE 3: INVENTARIO PLC — ESTRUCTURA DE SPRINTS

Sprint 0: Definición de Estándar, Alcance y Plantilla de Inventario

Objetivo: Establecer las reglas de extracción, la estructura del dataset de entrada y la plantilla maestra del inventario de tags.

Entregables:

Definición del método de extracción de tags según el PLC (ej. exportación .CSV / .L5X en Studio 5000 o equivalente).

Diseño del esquema de datos/columnas de la tabla del inventario (Tag Raw, Data Type, Scope, I/O Address, Área, Descripción Original).

Criterios de clasificación inicial y filtrado (Tags de Sistema vs. Tags de Proceso / E/S Físicas vs. Variables Internas).

Sprint 1: Extracción y Consolidación de Datos "Raw" (Sin Procesar)

Objetivo: Exportar y unificar la totalidad de tags de los controladores de todas las áreas del ingenio en un único depósito de datos.

Entregables:

Compilación de archivos fuente exportados por área/PLC (Trapiche, Clarificación, Evaporación, Cocimiento, Calderas, etc.).

Tabla unificada "Raw Tags" con el conteo total de variables extraídas.

Mapeo preliminar de volumen de data types (`BOOL` , `REAL` , `INT` , `UDT` , etc.).

Sprint 2: Diagnóstico, Limpieza y Detección de Anomalías

Objetivo: Auditar la calidad de los datos extraídos para identificar incoherencias y preparar la información para el filtrado.

Entregables:

Identificación y catalogación de Unused Tags (tags declarados en el PLC que no se usan en la lógica).

Reporte de tags sin descripción o con descripciones ambiguas/incompletas.

Detección de duplicados, alias colisionados o inconsistencias en nombres.

Sprint 3: Contextualización por Área de Proceso y Hardware (E/S)

Objetivo: Vincular cada tag a su ubicación física real dentro del ingenio y a su canal de E/S (I/O Mapping).

Entregables:

Clasificación estricta de tags por Área/Unidad operativa del Ingenio La Florida.

Mapeo de dirección de hardware (Rack / Slot / Channel) para variables físicas (4–20 mA, digitales, etc.).

Separación explícita de señales de campo (I/O Físicas) vs. variables puramente de software/HMI.

Sprint 4: Consolidación de la Base de Datos Baseline (Línea Base)

Objetivo: Cerrar el dataset definitivo del Inventario PLC del Ingenio La Florida y dejarlo 100% preparado para ser consumido por el script de Python y la Base de Datos Local en la Fase 4.

Entregables:

Dataset maestro del Inventario PLC finalizado (formato tabular limpio `.CSV` / `.XLSX`).

Matriz de métricas finales (total de tags activos de proceso, total por área, total por tipo de dato).

Validación de cierre de la Fase 3 y readiness para la Fase 4.

¡Decime cómo querés que abordemos o arranquemos el **Sprint 0**!

okey. iniciemos el sprint0. normalmente los "bloques" que decimos acá en el ingenio, son los programas en formato `.L5X` de studio5000. como hacemos que con eso se pasen las variables? recuerda que no puedo tocar much

SPRINT 0: Definición de Estándar y Extracción Segura

1. Seguridad y Viabilidad Operativa

Estando en plena molienda, es regla de oro no intervenir la lógica ni alterar los procesadores. La gran ventaja de Studio 5000 es que trabajar con archivos `.L5X` es un proceso completamente pasivo y seguro.

El archivo `.L5X` se puede generar abriendo el último respaldo (archivo `.ACD`) de manera offline y utilizando la función "Guardar como". Si no hay un respaldo reciente, se puede hacer la misma acción estando online conectado al PLC; esto simplemente guarda una copia del estado actual del proyecto en la computadora, sin causar micro-cortes, ni detener el ciclo de scan, ni requerir una descarga (download) al procesador. Los equipos seguirán trabajando sin enterarse.

2. Anatomía del Archivo `.L5X`

Estos "bloques" `.L5X` son estructuralmente archivos de texto plano escritos en formato XML. Esto es una ventaja inmensa para tu roadmap, ya que encaja a la perfección con la Fase 4. En lugar de copiar variables a mano, utilizaremos la estructura XML donde todos los tags están ordenados en nodos (específicamente dentro de las etiquetas `<Tags>`).

3. Plantilla de Inventario (Target de Extracción)

Para sentar las bases de la base de datos, definiremos que nuestro inventario capturará exactamente los siguientes atributos de cada variable dentro del `.L5X` :

Name (Raw Tag): El nombre actual tal como fue programado (ej. `Valvula_Jugo_01`).

TagType: Identifica si es un tag `Base` (memoria interna) o un `Alias` (clave para encontrar a qué canal físico de E/S está conectado).

DataType: El tipo de dato (`REAL` , `BOOL` , `DINT` o `UDT` corporativos).

Description: El comentario o texto original del programador, fundamental para mapear el tag viejo con el nuevo nombre ISA-5.1.

Scope: Si la variable pertenece al ámbito global (Controller Tags) o si está encapsulada dentro de uno de los bloques/programas (Program Tags).

4. Criterios de Filtrado Inicial

Para evitar que nuestra base de datos se llene de "basura", en este Sprint 0 dejamos establecido que filtraremos los tags de sistema. Ignoraremos parámetros internos de configuración de módulos de red o variables temporales de uso exclusivo del procesador, concentrándonos estrictamente en las variables de proceso y lazos de control.

Para avanzar hacia el **Sprint 1 (Extracción y Consolidación)**: ¿Prefieres que preparemos un pequeño script preliminar en Python para que extraiga automáticamente esta información leyendo tus archivos `.L5X`, o prefieres utilizar la opción "Tools > Export > Tags and Logic Comments" de Studio 5000 para sacar todo directamente en un archivo `.CSV` ?

recien vino un supervisor y le comentamos sobre el proyecto este de taggear las variables. y como hay un perosnal de IT con el que hablan los dos, entre el chico de it y el supervisor, el le dice que con pickup3 pare

¡Quedate totalmente tranquilo! Te entendí a la perfección. Es súper común que cuando la gente de OT (Automatización/Proceso) habla con la gente de IT (Sistemas), haya un choque de vocabularios técnicos y parezca que hablan en otro idioma.

Vamos a "traducir" lo que te dijeron y te doy mi opinión profesional, porque en realidad **lo que te propusieron es un golazo y nos adelanta muchísimo trabajo para la Fase 4.**

Aquí va la traducción de lo que probablemente escuchaste:

1. El famoso "pickup3"

Lo que el chico de IT quiso decir casi con un 100% de seguridad es `pycomm3` (se pronuncia pai-com-tres).

¿Qué es? Es una de las librerías de Python más famosas y utilizadas a nivel mundial para comunicarse específicamente con los PLC de la línea Logix de Allen-Bradley (ControlLogix y CompactLogix).

¿Qué hace? Exactamente lo que te describió el supervisor: te permite conectarte al PLC por red, preguntarle qué versión de firmware tiene, qué IP tiene, y lo más importante: **puede extraer la lista completa de todos los tags que están adentro del PLC** y guardarlos en un archivo `.txt` o `.csv`.

2. El misterioso "openclaw"

Acá hay dos opciones muy probables de lo que escuchaste al pasar:

Opción A ("Es Open Source"): Probablemente el de IT dijo que `pycomm3` es una herramienta Open Source (de código abierto/gratuita). Al escucharlo rápido en medio del ingenio, suena muy parecido a "openclaw".

Opción B ("OPC UA"): (Se pronuncia O-Pe-Ce U-A). Es un protocolo de comunicación industrial estándar que la gente de IT ama usar para sacar datos de los PLC hacia bases de datos.

Mi opinión técnica: ¿Sirve esto para la Fase 3?

¡Sí, es una noticia excelente! De hecho, apruebo totalmente que lo hagan por esta vía por tres motivos fundamentales:

Es 100% pasivo y seguro: Usar `pycomm3` para extraer variables no interrumpe el ciclo de scan del PLC. Solo "lee" datos a través de la red (EtherNet/IP). La molienda y los lazos de control seguirán funcionando sin enterarse de que alguien está copiando los nombres de los tags. No hay riesgo de detener la planta.

Se alinea con tu Fase 4: Como en la Fase 4 tenés planeado usar **Python y Base de Datos**, el hecho de que IT ya maneje Python (`pycomm3`) significa que tienen la plataforma lista. Ya hablan el mismo idioma que tu hoja de ruta.

Nos ahorra entrar al Studio 5000: Si IT puede correr ese script y pasarte un archivo `.txt` o `.csv` con todos los tags de la red, ya no necesitás conectarte vos al PLC ni buscar los archivos `.L5X`. Ellos te hacen la extracción y nosotros empezamos directamente a limpiar y estructurar los datos (nuestro Sprint 2).

¿Cómo lo integramos a nuestro Sprint 0?

Nuestro plan del Inventario no cambia, solo cambia de dónde sacamos los datos. El objetivo sigue siendo conseguir esa lista de variables (Raw Tag, DataType, Descripción, etc.).

Para seguir avanzando de forma segura, ¿preferís que dejemos que IT corra su programa y nos entregue ese archivo `.txt` para que lo analicemos nosotros, o querés que igualmente aseguremos una copia del archivo `.L5X` a la antigua por si lo de IT se demora?

bien. entoces si puede ser pycomm3 entonces para pedir la info del plc. pero openclaw si está bien escrito, pq lo conozco. OpenClaw es un agente personal de inteligencia artificial de código abierto que se ejecuta localme

¡Qué excelente aclaración! Tienes toda la razón y es una aproximación técnica de altísimo nivel por parte del equipo de IT. Al no ser una herramienta tradicional del ámbito puramente industrial, no la asocié inmediatamente, pero tu descripción es impecable.

Integrar **OpenClaw** con `pycomm3` representa una arquitectura de automatización de datos sumamente avanzada y encaja a la perfección con el esquema de la Fase 4 de tu proyecto (Python + Base de Datos).

Análisis de la Arquitectura Propuesta (IT + OT)

Básicamente, el equipo de IT está armando un flujo de extracción autónomo:

El Motor de Extracción (`pycomm3`): Será la librería de Python encargada de la comunicación industrial. Se conectará a la IP del ControlLogix mediante el protocolo EtherNet/IP y extraerá la lista de variables, tipos de datos y descripciones sin interrumpir el proceso de molienda.

El Agente Orquestador (OpenClaw): Actuará como el administrador automatizado de la tarea. En lugar de que un operador ejecute el script manualmente, OpenClaw puede ejecutar el código de `pycomm3`, tomar los datos crudos, formatearlos en un archivo de texto o `.csv`, guardarlos en tu sistema de archivos e incluso notificarte por WhatsApp o Telegram que el inventario está listo para descargar.

Impacto en nuestro Sprint 0 y Sprint 1

Esta metodología nos simplifica enormemente el trabajo de la Fase 3:

Elimina el riesgo manual: Ya no es necesario acceder a Studio 5000 ni manipular archivos `.L5X` manualmente, mitigando cualquier temor de afectar los procesadores en funcionamiento.

Automatización de la Línea Base: La extracción de datos crudos (Raw Data) queda completamente delegada al agente de inteligencia artificial de IT.

Siguiente Paso Estratégico

Para garantizar que el archivo generado por OpenClaw nos sirva directamente para el filtrado y análisis del **Sprint 2**, necesitamos definir los parámetros exactos de salida.

¿Deseas que estructuramos ahora mismo el listado exacto de campos obligatorios (Nombre del Tag, Tipo de Dato, Alias, Descripción, Scope) para entregárselo al equipo de IT como requisito formal de extracción?

bien. supongamos por un momento que no puedo usar openclaw y debo hacerlo manualmente a la obtención de datos de variables de un plc. quiero hacer una prueba aquí en el taller. tenemos un plc nosotros y podría hac

¡Totalmente posible y sumamente recomendable! Hacer una prueba previa en el taller con el PLC que tienen a disposición es la mejor estrategia antes de tocar o relevar los equipos de producción de la planta. Además, hacerlo por tus propios medios te da un control total del proceso sin depender de licencias de APIs ni costos de terceros.

Para lograr una solución **100% gratuita, local y totalmente bajo tu control**, podemos prescindir de OpenClaw y ejecutar tú mismo el proceso. Tienes dos caminos manuales/locales muy sencillos para extraer los datos:

Opción A: A través de un Script de Python Local (Usando `pycomm3`)

Como `pycomm3` es una librería de Python de código abierto y totalmente gratuita, no necesitas ninguna API Key ni conexión a Internet. Solo necesitas que tu computadora y el PLC de prueba estén en la misma red de red Ethernet.

¿Cómo funciona el proceso?

Instalas Python en tu computadora local.

Instalas la librería mediante el comando: `pip install pycomm3` .

Ejecutas un script de pocas líneas donde especificas la dirección IP del PLC de tu taller.

El script se conecta al PLC, lee la estructura interna de la memoria (Controller Tags) y guarda automáticamente un archivo `.csv` o `.txt` con todos los nombres de los tags, sus tipos de datos (`BOOL` , `REAL` , `DINT`), sus descripciones y los alias.

(Si te parece bien, en el siguiente paso te puedo armar el código completo de Python listo para copiar, pegar y ejecutar en tu prueba de taller).

Opción B: Exportación Directa desde Studio 5000 / RSLogix 5000

Si prefieres no usar código por ahora y hacerlo de forma 100% nativa con las herramientas de Rockwell:

Te conectas al PLC de taller con Studio 5000 (o abres el proyecto `.ACD` de prueba).

Vas al menú superior: **Tools** → **Export** → **Tags and Logic Comments...**

Guardas el archivo resultante en formato `.CSV` .

Ese archivo `.csv` contendrá absolutamente toda la base de datos de tags del PLC, lista para abrirse en Excel o procesarse más adelante.

Plan de Acción para la Prueba de Taller

Para cerrar formalmente este **Sprint 0** y pasar al **Sprint 1 (Prueba de Extracción Real)**:

Definir la IP del PLC de taller: Verificar la dirección IP del procesador de prueba.

Elegir la vía de extracción: Decidir si probamos con la exportación nativa a `.csv` de Studio 5000 o con el script propio de Python (`pycomm3`).

¿Cuál de los dos métodos prefieres que preparemos para realizar la prueba en el taller?

hay un tema. me dicen que de la forma que tienen organizadas las variables, hay variables que usan en un bloque como salida pero que se utilizan como entrada en otro bloque, donde recién si se usa la variable. ta

Las imágenes que envías aclaran **absolutamente todo** y le dan sentido exacto a lo que te pidieron el ingeniero y el supervisor.

Mirando la segunda imagen (el diagrama de bloques FBD), se ve con total claridad el problema técnico al que se enfrentan:

Diagnóstico del Problema Técnico

Variables de Salida Intermedias vs. Variables de Proceso Reales:

En la imagen se ve una rutina de escalado donde la salida del bloque (ejemplo: 33.11) se conecta a la variable `EVAP_S13_LCV_ENT_JG_A_CAJA_2`.

El problema de hacer un "dump" o exportación simple del CSV: Studio 5000 te exporta todas las variables, incluyendo auxiliares, auxiliares de horómetros (`HORAS_CAJA_2` , `MINUTOS_CAJA_2`), bits de habilitación (`BNOT_30`), variables de estado interno, etc.

Si hay 32 PLCs con miles de variables, terminarías con **decenas de miles de tags basuras o auxiliares** que no son verdaderos instrumentos de campo (ISA-5.1).

Cadenas de Variables (Entradas que son Salidas de otros bloques):

Muchas variables nacen en una rutina de escalado (`FAB_ESCALADOS`), se guardan en un Tag, y luego ese Tag entra a otro bloque en la rutina de evaporación (`FAB_EVAP`) para ser procesado.

A mano es inviable seguir el rastro de dónde nace y dónde muere cada variable en 32 PLCs.



¿Qué es lo que realmente necesita el proyecto?

Lo que el ingeniero quiere es **filtrar la paja del trigo**:

Identificar únicamente las **Variables de Proceso de Verdad (Tags Físicos o Principales)**.

Descartar los tags intermedios o salidas que no tienen un destino relevante de proceso (como contadores auxiliares, habilitaciones o bloques lógicos intermedios `BNOT`).



Plan Estratégico de Acción (Probar con `FAB_ESCALADOS`)

Hacer la prueba piloto limitada a la rutina `FAB_ESCALADOS` es la decisión más inteligente del proyecto. Nos permite validar la lógica sin abrumarnos.

Para procesar este caso sin volvernos locos a mano, el enfoque debe ser el siguiente:

Paso 1: Analizar qué nos da el CSV que ya exportaste

El archivo `.csv` de Studio 5000 nos da la lista de **declaración de Tags** (Nombre, Tipo de Dato, Comentario), pero **no nos dice cómo están interconectadas** en el dibujo del bloque (FBD).

Paso 2: Por qué un Script (o herramienta) es la única solución real

Como los bloques FBD unen líneas visualmente, la única forma de saber si una salida de un bloque entra a otro o muere en un indicador es **analizar la rutina completa**.

Si exportamos la rutina `FAB_ESCALADOS` en formato `.L5X` (que es un archivo de texto/XML que representa las conexiones del dibujo) o usamos `pycomm3` , podemos escribir un script en Python que responda automáticamente estas preguntas:

¿Este Tag está conectado a la **salida de un bloque de escalado** (ej. SCL, SCP, Alarma)?

¿A qué otro bloque o rutina va a parar ese Tag como **entrada**?

¿Tiene descripción o comentario asociado?

La Propuesta Concisa para Comenzar

No hagamos código todavía. Aseguremos la estrategia para el piloto de `FAB_ESCALADOS` :

Definir la "Variable Relevante":

Consideraremos como Tag candidato a renombra/tagear bajo ISA-5.1 únicamente a las variables que estén conectadas a las salidas de escalado de señales de campo (`PV_OUT` , `SCL` , etc.) como la que se ve en la imagen:

`EVAP_S13_LCV_ENT_JG_A_CAJA_2` .

Categorizar las "Salidas Auxiliares":

Lógicas como `HORAS_CAJA_2` o `MINUTOS_CAJA_2` se marcarán automáticamente como **Variables de Diagnóstico/Mantenimiento**, no como instrumentos directos de proceso.

Insumo de Prueba:

Para analizar `FAB_ESCALADOS` , necesitaremos:

El archivo `.CSV` que ya exportaste de ese PLC.

(Opcional) Exportar únicamente el programa o rutina `FAB_ESCALADOS` dando clic derecho en Studio 5000 → Export Component (lo guarda como `.L5X`).

¿Te parece bien que utilicemos el `.CSV` que ya tienes generado de `FAB_ESCALADOS` para analizar qué información nos da? entonces, deberíamos modificar los sprints de la fase 3 o no? bien. ya tengo el csv y el l5x. ahora que hacemos?

¡Exacto! Teniendo la realidad de la planta sobre la mesa, **ajustar los Sprints de la Fase 3 es lo más profesional y adecuado.**

Modificar el alcance para adaptarlo a la arquitectura real del software (diagramas FBD, rutinas de escalado `FAB_ESCALADOS` y filtrado de tags basura/auxiliares) nos garantiza no perder tiempo procesando miles de variables inservibles.



Ajuste Oficial de los Sprints — FASE 3: INVENTARIO PLC

Sprint 0: Alcance, Criterios de Selección y Piloto `FAB_ESCALADOS` ✓ (Concluido: estrategia de trazabilidad FBD y prueba acotada).

Sprint 1: Extracción y Parseo de Rastro FBD (`.CSV` + `.L5X`) 📌 (ESTAMOS ACÁ)

Extraer las conexiones reales entre bloques de la rutina `FAB_ESCALADOS` .

Vincular salidas de escalado de proceso con sus tags de destino.

Sprint 2: Clasificación de Variables y Filtrado de Tags Auxiliares

Separar Tags Físicos / Principales de variables de diagnóstico (`HORAS_CAJA_2` , `MINUTOS_CAJA_2` , bits internos `BNOT`).

Sprint 3: Matriz de Trazabilidad de Lazos (Entradas → Escalado → Proceso)

Construir la tabla limpia donde cada variable de proceso tiene identificada su origen y su rutina de uso.

Sprint 4: Escalamiento del Algoritmo a los 32 PLCs del Ingenio

Validar la regla de extracción para aplicarla masivamente en las demás rutinas y procesadores.



SPRINT 1: ¿Qué hacemos ahora con el `.CSV` y el `.L5X` ?

Como ya tienes los dos archivos de la prueba de `FAB_ESCALADOS` , el objetivo directo de este **Sprint 1** es **cruzar la información**:

El `.CSV` nos da la "Ficha Técnica" de la variable:

Nombre del Tag, Comentario/Descripción original, DataType (`REAL` , `BOOL` , etc.) y Scope.

El `.L5X` nos da la "Conectividad Rastro" del dibujo FBD:

Nos dice exactamente qué Tag está colgado a la salida de un bloque de escalado (`PV_OUT` / `SCL`) y si ese Tag luego viaja como entrada hacia otra rutina (ej. de `FAB_ESCALADOS` a `FAB_EVAP`).



Paso Práctico a Seguir

Para que analicemos y hagamos la prueba sin que expongas archivos enteros o pesados:

Abre el archivo `.CSV` con Bloc de notas o Excel:

Busca y copia 3 o 4 filas representativas donde aparezcan variables de escalado de la caja o evaporación (como la de tu foto:

`EVAP_S13_LCV_ENT_JG_A_CAJA_2`).

Abre el archivo `.L5X` con Bloc de Notas (o VS Code / Notepad++):

El `.L5X` es puro texto estructurado (XML).

Busca con `Ctrl + F` la variable `EVAP_S13_LCV_ENT_JG_A_CAJA_2` o el bloque `HORIMETRO_03` y copia ese pequeño fragmento/bloque de texto donde aparece esa variable.

Pega esos dos fragmentos cortos por acá.

Con ese ejemplo real en mano, analizaremos exactamente qué etiquetas del `.L5X` debemos leer con un script para automatizar la extracción de **únicamente las variables**

de proceso reales y descartar los auxiliares.

ahi te comparto el csv y el l5x. me comentan acá que ya hicieron un programita que lee pasandole la ip del plc, obteniendo la info que hablabamos, y obteniendo los tags y eso lo guardaba en un excel. el tema es q no le e

¡Hola! Entiendo perfectamente el problema que están teniendo. Estuve revisando el contenido que me compartiste y encontré la razón exacta por la cual el Excel no les está mostrando la tabla con los tags.

¿Qué está pasando realmente?

Confusión de formato (CSV vs. XML): Aunque mencionas que el archivo se llama `FABRICA_FAB_ESCALADOS_Tags.CSV`, el texto que me pasaste comienza con la etiqueta `<?xml version="1.0" encoding="UTF-8" standalone="yes"?>` y la cabecera `<RSLogix5000Content>`. Esto indica que es **código XML puro** (específicamente, un formato de exportación `.L5X` de RSLogix/Studio 5000), no un archivo separado por comas (CSV).

Desconocido

Comportamiento de Excel: Si Excel intenta abrir un archivo `.CSV` que por dentro contiene código XML, no sabrá cómo acomodar los datos en columnas. Simplemente volcará todo el código crudo como texto en las celdas, haciendo que sea imposible ver una lista ordenada.

Estructura del contenido: El fragmento del archivo que me compartiste no es una tabla plana. Contiene la configuración jerárquica interna del PLC, como las definiciones de tipos de datos (`<DataTypes>`), los módulos de hardware (`<Modules>`) y los bloques de instrucciones (`<AddOnInstructionDefinitions>`). La lista de tags propiamente dicha (la sección `<Tags>`) probablemente esté más abajo en el documento, pero está envuelta en este lenguaje de marcado, no en formato de tabla.

Desconocido + 2

¿Cómo solucionarlo?

Para que obtengan el resultado que buscan, tienen dos caminos principales:

Modificar el código del "programita" (Recomendado): Si el objetivo es tener un Excel limpio (con columnas como Nombre del Tag, Tipo de Dato, Valor, etc.), el

programa no debería guardar la respuesta cruda en formato L5X/XML. El script debería leer los tags del PLC, armar una tabla de datos en memoria (por ejemplo, usando listas en C# o `pandas` en Python) y luego exportar esa tabla específicamente como un archivo `.csv` real o `.xlsx`.

Cambiar la extensión del archivo: Si no pueden modificar el programa ahora mismo, cámbienle la extensión al archivo de `.CSV` a `.XML`. Cuando lo abran con Excel, el programa les preguntará cómo desean abrirlo (elijan "Como una tabla XML"). Sin embargo, tengan en cuenta que al ser un archivo estructurado para Studio 5000, Excel generará muchísimas columnas vacías y será un poco difícil de leer.

¿En qué lenguaje de programación escribieron el programita para extraer los datos así les ayudo a corregir el código y exportar el Excel correctamente?

no. los archivos csv y xml los obtuve yo hoy. pero ayer mi compañero de it ya habia obtenido info a rtraves de un programa en python. y le pasamos una ip de un plc y le dio el excel que te pasé, o sea son dos resultados. el excel es de el, y los otros dos son mios

¡Ah, entiendo perfectamente la confusión! Gracias por la aclaración.

Entonces tenemos dos resultados distintos en juego:

Tus archivos de hoy: El archivo que vimos antes (`FABRICA_FAB_ESCALADOS_Tags.CSV`), que efectivamente tiene estructura XML por dentro y por eso Excel no lo puede acomodar en columnas.

El Excel de tu compañero: El que generó ayer usando Python y conectándose directamente a la IP del PLC.

El diagnóstico anterior explicaba por qué tu archivo no se veía como tabla. Si el Excel de tu compañero de IT tampoco está mostrando la información que esperan, el problema ahí debe ser distinto (quizás un tema en cómo el script de Python está parseando la respuesta del PLC antes de armar el archivo).

¿Me podrías compartir el Excel que generó tu compañero o el código de Python que usó, para que pueda analizar directamente dónde se están perdiendo los tags?

¡Aclaradísimo! Ahora entiendo cómo llegamos a estos dos resultados distintos.

Lamentablemente no puedo abrir el archivo `inventario_plcs_20260722_090421_2.xlsx` directamente, pero basándome en el fragmento de código en Python que compartiste, queda bastante claro por qué ese Excel no contiene la lista de variables que esperaban.

Aquí te detallo lo que está haciendo ese script:

1. Solo hace "Descubrimiento" e "Identidad"

El código utiliza la librería `pycomm3` para hacer un reconocimiento o escaneo de la red. Las funciones principales que vemos ahí (`discover_devices` y `check_single_device`) usan comandos de red para detectar dispositivos. Estos comandos se limitan a preguntar "¿Quién está ahí?", y el módulo de comunicaciones del PLC responde únicamente con sus datos básicos de identidad: la IP, el fabricante (Vendor) y el tipo de producto (Product Type).

PY + 2

2. El nombre del archivo y los comentarios coinciden

El hecho de que el archivo generado se llame `inventario_plcs_20260722_090421_2.xlsx` tiene todo el sentido, ya que el script está armando un **inventario** de equipos conectados a la red, no volcando su base de datos interna.

De hecho, dentro de la función `filter_logix_devices` , hay un comentario clave escrito en el código que dice: "el intento real de conexión (`get_tag_list`) es el que termina de confirmar si es un Logix".

Esto nos confirma que descubrir el dispositivo es solo el primer paso. Extraer las variables requiere un segundo paso explícito (`get_tag_list`) que probablemente no se está exportando a ese Excel de inventario.

PY + 1

En resumen:

Tus archivos (CSV/XML): Sí tienen los tags en su interior, pero están encapsulados en código XML que Excel no sabe convertir a columnas de forma amigable.

El Excel de tu compañero de IT: Está perfectamente formateado, pero su programa de Python se limitó a extraer la identidad del PLC (como su IP y modelo) en lugar de leer las variables de memoria.

PY

La solución ideal:

Dado que tu compañero ya tiene Python y la librería `pycomm3` funcionando, la mejor alternativa es que él modifique ligeramente su script. Una vez que detecta la IP, en lugar de quedarse solo con la "identidad", el script debería abrir una conexión con el PLC usando `LogixDriver(ip)` (en lugar de `CIPDriver`) y llamar a `.get_tag_list()`. De esa forma, Python podrá generar un Excel limpio y ordenado con todos los tags, combinando lo mejor de los dos mundos.

ahora estoy por trabajar con claude code pro así que necesito un prompt maestro para que sirva de contexto de todo este chat y proyecto, de los inconvenientes que tenemos por la cantidad de variables, viendo los aprox 32 plc s

Es un desafío monumental encarar la estandarización de 20,000 variables distribuidas en 32 PLCs. La presión del tiempo es real, considerando que el reloj ya marca las 10:41 AM y el objetivo es tener un caso de éxito sólido (el primer PLC) listo para el mediodía. Un volumen de este calibre requiere una estrategia impecable para evitar cuellos de botella y errores de migración.

Para que Claude Code asuma el rol de auditor y ejecutor técnico con total precisión, he diseñado un "Prompt Maestro" que sintetiza las directrices de tu proyecto, las reglas de la norma ANSI/ISA-5.1-2024, y el contexto de Allen Bradley (Studio 5000).

Copia y pega el siguiente bloque de texto directamente en tu sesión con Claude Code:

Prompt Maestro para Claude Code Pro

Rol y Contexto General

Actúa como un Ingeniero Senior de Automatización y Auditor Experto en la norma ANSI/ISA-5.1-2024 y sistemas Rockwell (Studio 5000).

Estás liderando técnicamente el proyecto de "Estandarización del Ingenio La Florida". Nuestro desafío actual es masivo: tenemos aproximadamente 32 PLCs que contienen alrededor de 20,000 tags o variables. Necesitamos organizar, auditar y renombrar esta base de datos sin afectar la operatividad, unificando criterios entre Mantenimiento, Operaciones e Ingeniería.

Objetivos de la Sesión (Sprint Actual: 10:40 AM a 12:00 PM)

1. **Auditoría y Estrategia:** Analizar nuestro enfoque y dar recomendaciones arquitectónicas sobre cómo encarar el renombrado masivo de 20,000 variables de forma segura y automatizada.

2. **Prueba de Concepto (POC):** Auditar y renombrar los tags de **1 solo PLC piloto** antes del mediodía para validar la metodología.

Marco Normativo y Reglas de Nomenclatura

La norma **ANSI/ISA-5.1-2024** se utilizará exclusivamente para instrumentos físicos, funciones de instrumentación y funciones de control de proceso. Las variables internas del PLC seguirán una convención corporativa propia.

Estructura Obligatoria del Tag Base: [ÁREA]_[VARIABLE][FUNCIÓN]_[NÚMERO]

- **Área:** 3 dígitos (ej. 100 Molienda, 200 Destilería).
- **Variable/Función:** Según diccionario ISA-5.1 (ej. PT, LIT, FIC).
- **Número:** Secuencial de 3 o 4 dígitos (ej. 001).
- Ejemplo final: 100_PT_001

Reglas para Studio 5000 (Allen Bradley):

- Se deben utilizar **UDTs (User Defined Data Types)**. Ejemplo: `Transmisor_Analogo` con miembros `.PV`, `.Sim`, `.Fault`.
- El Tag base en el controlador debe ser el Tag ISA funcional.
- Las entradas físicas usarán Aliasing vinculando el Tag de la tarjeta física con el Tag funcional.

Resumen del Diccionario Corporativo (Restricciones ISA-5.1):

- **P (Pressure):** Presión o Vacío. Restricción: No usar para "Punto de prueba" como primera letra. Visores locales son PG, no PI.
- **T (Temperature):** Temperatura. Restricción: Visores locales son TG. Modificador TS es de seguridad crítica.
- **L (Level):** Nivel. Restricción: No usar para alarmas luminosas (usar A). L como modificador en válvulas significa "Cerrado".
- **F (Flow):** Caudal. Restricción: Integradores son FQ. Diferencial es FD, no PD.
- **A (Analysis / Alarm):** Análisis (primera letra) o Alarma (letra sucesiva). Restricción: El componente analizado (ej. pH) va fuera de la burbuja en P&IDs.
- **Q (Quantity):** Cantidad o Totalizador. Restricción: No confundir con Caudal (F).
- **H (Hand / High):** Acción Manual (primera letra) o Límite Alto (modificador).
- **I (Current / Indicate):** Corriente eléctrica (primera letra) o Indicación SCADA/HMI (letra sucesiva).
- **V (Vibration / Valve):** Vibración (primera letra) o Válvula de control (letra sucesiva). Restricción: CV es solo para válvulas autoaccionadas.
- **D, M, C (User's Choice):** Definidos localmente como Densidad, Humedad y Conductividad/Control, respectivamente.
- **Z (Position / SIS):** Posición (primera letra) o Sistema Instrumentado de Seguridad (modificador crítico).

- **E (Voltage / Element):** Voltaje (primera letra) o Elemento primario/Sensor (letra sucesiva).
- **J (Power / Scan):** Potencia (primera letra). Prohibición estricta: JAMÁS usar J para "Jugo".

Tu Tarea Inmediata:

1. Confirma que entiendes el contexto y las reglas.
2. Proporciona una recomendación de 3 pasos para abordar este proyecto de 20,000 tags de forma iterativa y segura.
3. Solicítame el listado crudo (CSV, Excel o texto) de los tags de nuestro "PLC Piloto" para comenzar la depuración y aplicar las reglas de nomenclatura inmediatamente.

Para agilizar el trabajo con el código del PLC piloto de cara al mediodía, ¿tienes el listado actual de las variables de ese primer equipo exportado en formato CSV, o planeas extraerlas directamente desde el PLC?

añadele que hay una carpeta AUTOMATISMO_AGUSTIN y dentro de files, esta el programa de python que hice sobre la asignacion de tags. hazlo mas extenso cosa de contar bien todo el contexto completo de lo que busco con esto

Entiendo perfectamente. Añadir el contexto del script en Python es clave, ya que le dará a Claude Code (que tiene acceso al sistema de archivos) la capacidad de leer tu código, entender tu lógica actual y ayudarte a escalarla o corregirla directamente.

He ampliado el "Prompt Maestro" para detallar la magnitud del proyecto en el Ingenio La Florida, la existencia de tu herramienta personalizada y cómo encaja todo en la estrategia de migración.

Copia y pega este nuevo bloque en tu sesión con Claude Code:



Prompt Maestro Expandido para Claude Code Pro

Rol y Contexto General del Proyecto

Actúa como un Ingeniero Senior de Automatización, Desarrollador Python y Auditor Experto en la norma ANSI/ISA-5.1-2024 y sistemas Rockwell (Studio 5000).

Estás asumiendo el liderazgo técnico del proyecto crítico de "Estandarización del Ingenio La Florida". El panorama general es monumental: debemos migrar, organizar y estandarizar aproximadamente 20,000 variables distribuidas a lo largo de 32 PLCs. El objetivo final de este proyecto no es solo renombrar tags, sino unificar definitivamente los criterios de trabajo entre los departamentos de Mantenimiento, Operaciones e Ingeniería, creando una arquitectura de datos escalable, robusta y a prueba de errores.

Entorno de Trabajo y Herramientas Propias

Para evitar hacer este trabajo de forma manual y mitigar errores humanos, he comenzado

a desarrollar una solución automatizada. En tu entorno de trabajo actual, existe un directorio llamado `AUTOMATISMO_AGUSTIN` y, dentro de este, una subcarpeta llamada `files`. Allí encontrarás un programa en Python que he creado específicamente para manejar la asignación lógica de estos tags.

- **Tu rol con esta herramienta:** Deberás ser capaz de leer este script, entender la lógica que he intentado implementar, auditarla bajo las reglas normativas y ayudarme a ejecutarla o mejorarla para procesar los lotes de variables.

Objetivos de la Sesión (Sprint Actual: Antes del Mediodía)

1. **Análisis del Entorno:** Leer y comprender el código Python existente en `AUTOMATISMO_AGUSTIN/files/` para entender mi enfoque actual de asignación de tags.
2. **Auditoría y Estrategia:** Definir la arquitectura de cómo este script de Python interactuará con las exportaciones de Studio 5000 (archivos CSV/L5X) para renombrar las 20,000 variables de forma segura.
3. **Prueba de Concepto (POC) Crítica:** Auditar y procesar los tags de **1 solo PLC piloto** antes del mediodía. Necesitamos un caso de éxito inmediato que valide que el script de Python y las reglas de nomenclatura funcionan en conjunto.

Marco Normativo y Reglas de Nomenclatura

La norma **ANSI/ISA-5.1-2024** se utilizará exclusivamente para instrumentos físicos, funciones de instrumentación y funciones de control de proceso. Las variables internas del PLC seguirán una convención corporativa propia.

Estructura Obligatoria del Tag Base: `[ÁREA]_[VARIABLE][FUNCIÓN]_[NÚMERO]`

- **Área:** 3 dígitos (ej. 100 Molienda, 200 Destilería).
- **Variable/Función:** Según diccionario ISA-5.1 (ej. PT, LIT, FIC).
- **Número:** Secuencial de 3 o 4 dígitos (ej. 001).
- Ejemplo final: `100_PT_001`

Reglas para Studio 5000 (Allen Bradley):

- Se deben utilizar **UDTs (User Defined Data Types)**. Ejemplo: `Transmisor_Analogo` con miembros `.PV`, `.Sim`, `.Fault`.
- El Tag base en el controlador debe ser el Tag ISA funcional.
- Las entradas físicas usarán Aliasing vinculando el Tag de la tarjeta física con el Tag funcional.

Resumen del Diccionario Corporativo (Restricciones ISA-5.1):

- **P (Pressure):** Presión o Vacío. Restricción: No usar para "Punto de prueba" como primera letra. Visores locales son PG, no PI.
- **T (Temperature):** Temperatura. Restricción: Visores locales son TG. Modificador TS es de seguridad crítica.
- **L (Level):** Nivel. Restricción: No usar para alarmas luminosas (usar A). L como modificador en válvulas significa "Cerrado".

- **F (Flow):** Caudal. Restricción: Integradores son FQ. Diferencial es FD, no PD.
- **A (Analysis / Alarm):** Análisis (primera letra) o Alarma (letra sucesiva). Restricción: El componente analizado (ej. pH) va fuera de la burbuja en P&IDs.
- **Q (Quantity):** Cantidad o Totalizador. Restricción: No confundir con Caudal (F).
- **H (Hand / High):** Acción Manual (primera letra) o Límite Alto (modificador).
- **I (Current / Indicate):** Corriente eléctrica (primera letra) o Indicación SCADA/HMI (letra sucesiva).
- **V (Vibration / Valve):** Vibración (primera letra) o Válvula de control (letra sucesiva). Restricción: CV es solo para válvulas autoaccionadas.
- **D, M, C (User's Choice):** Definidos localmente como Densidad, Humedad y Conductividad/Control, respectivamente.
- **Z (Position / SIS):** Posición (primera letra) o Sistema Instrumentado de Seguridad (modificador crítico).
- **E (Voltage / Element):** Voltaje (primera letra) o Elemento primario/Sensor (letra sucesiva).
- **J (Power / Scan):** Potencia (primera letra). Prohibición estricta: JAMÁS usar J para "Jugo".

Tu Tarea Inmediata:

1. Confirma que entiendes el alcance del proyecto en el Ingenio La Florida y las normativas estipuladas.
2. Utiliza tus herramientas para explorar la ruta `AUTOMATISMO_AGUSTIN/files/`, lee el script de Python que se encuentra allí y hazme un breve resumen de lo que entiendes que hace el código actualmente.
3. Indica qué necesitas (ej. archivo CSV del PLC piloto) para que podamos hacer correr el script ajustado y tener el primer equipo estandarizado antes del mediodía.

¿Prefieres que Claude modifique el script de Python directamente si encuentra oportunidades de optimización para Studio 5000, o prefieres que solo te sugiera los cambios para que tú los apliques?

bien. ya le pase el prompt y me dijo: "El corazón del proyecto no es la GUI de altas manuales (sirve para tags nuevos), sino procesar en lote la exportación del PLC piloto. Para eso necesito de tu lado:

Entiendo perfectamente la situación. Es una duda técnica muy válida cuando se pasa de la teoría de la norma a la exportación cruda de un PLC, pero lo bueno es que tu manual ya tiene las bases exactas para responderle.

Aquí tienes una respuesta estructurada y técnica que puedes enviarle, basada estrictamente en las definiciones de tu documento:

1. Sobre la Regla de Numeración

Dile que el estándar del Ingenio apunta a una numeración basada en el lazo de control.

El número debe ser un secuencial de 3 o 4 dígitos asignado para el lazo de control completo.

PDF

La estructura jerárquica del Tag base que deben seguir es [ÁREA][VARIABLE]

[FUNCIÓN][NÚMERO] .

PDF

Por ejemplo, el tag `100_PT_001` agrupa el prefijo de área (100 para Molienda), las letras de identificación funcional (PT) y el número de lazo compartido (001).

PDF

(Nota para ti: El manual especifica que ciertas decisiones de numeración, como decidir si las válvulas de control usarán exactamente el mismo número que el lazo o una serie numérica aparte, deben terminar de definirse en el Estándar Corporativo. Si ya tienes una preferencia para esto, es un buen momento para sumarlo a tu respuesta).

PDF

2. Criterio Tag Viejo → Tag Nuevo (Físico vs Interno)

Para que el ingeniero pueda filtrar y distinguir rápidamente en su exportación de Studio 5000 qué es un instrumento de campo y qué es memoria interna, pásale estos tres criterios de tu estándar:

Por Aliasing (El método más directo en el .L5X): Para las entradas físicas reales, el estándar recomienda utilizar el Aliasing. Él podrá distinguirlos porque estarán vinculando el Tag de la tarjeta física de I/O (por ejemplo, `Local:1:I.Ch0Data`) directamente con el Tag funcional del instrumento (por ejemplo, `PT_101`).

PDF + 1

Por Nomenclatura Pura (Norma ISA): La norma ANSI/ISA-5.1 se utilizará exclusivamente para todo lo que represente una función real del proceso y los instrumentos físicos en sí (por ejemplo, `PT-101`, `TT-205`, `FIC-301`).

PDF

Por Sufijos o miembros de UDT (Variables Internas): Las variables de implementación o software interno no usan la norma ISA pura, sino una convención corporativa del PLC. Se distinguen porque llevan sufijos específicos (por ejemplo, `PT_101_RAW` , `PT_101_PV` , `PT_101_SIM` , `PT_101_FAULT`). Además, el estándar recomienda usar UDTs (User Defined Data Types), por lo que las variables internas se verán como miembros del tipo de dato analógico (ejemplo: `.PV` , `.Sim` y `.Fault`).

PDF + 2

Con esta respuesta le das a su herramienta de procesamiento en lote los patrones exactos (Alias, sufijos y miembros de UDT) que necesita buscar en el Excel o .L5X para mapear todo correctamente

genera un prompt maestro para pasarle la info a notebooklm en donde le cargare el pdf sobre la estandarizacion, el pdf de la norma isa y el trabajo q hago con google gemini y claude code

Aquí tienes un **Prompt Maestro** optimizado para **NotebookLM**. Este prompt está diseñado para establecer el rol, la jerarquía de las fuentes y las reglas de interacción para que la IA entienda perfectamente la relación entre la norma general, la regla corporativa de la planta y la automatización con IA que estás realizando.

Puedes pegarlo directamente en el chat de NotebookLM como **mensaje inicial de configuración** (o usarlo como guía para tus consultas).

Prompt Maestro para NotebookLM

Plaintext

[ROL Y PERFIL]

Actúa como un Ingeniero Senior de Automatización, Control e Instrumentación, experto en I

[CONTEXTO Y FUENTES CARGADAS]

He cargado en este cuaderno tres fuentes fundamentales que debes combinar y cruzar en tod

1. NORMA ISA (PDF): Representa el marco teórico internacional de simbología, identificaci
2. ESTÁNDAR DE ESTANDARIZACIÓN (PDF): Representa las reglas de negocio específicas corpor
3. TRABAJO Y WORKFLOW CON GEMINI / CLAUDE CODE (PDF): Representa los flujos de trabajo, s

[JERARQUÍA DE VERDAD Y REGLAS DE DECISIÓN]

A la hora de responder o analizar información, aplica estrictamente este orden de priorid

- Prioridad 1 (Regla Local): El ESTÁNDAR DE ESTANDARIZACIÓN prevalece sobre la norma gene
- Prioridad 2 (Marco Teórico): La NORMA ISA se utiliza para validar la función primaria d
- Prioridad 3 (Ejecución y Flujo): El TRABAJO CON GEMINI / CLAUDE CODE dicta el método pr

[OBJETIVOS DE TUS RESPUESTAS]

Cuando interactúe contigo, tus tareas principales serán:

1. Validar y reestructurar Tags: Evaluar si un tag (viejo o propuesto) cumple con la norma
2. Mapeo y Migración de Tags: Ayudarme a crear tablas de equivalencias entre el tag físico
3. Generación de Prompts e Instrucciones para Claude Code / Gemini: Redactar o perfeccionar
4. Resolución de Duda Técnicas: Responder consultas sobre cómo interpretar variables en S

[FORMATO Y TONO DE LAS RESPUESTAS]

- Idioma: Español.
- Tono: Técnico, directo, profesional y orientado a la ingeniería práctica.
- Estructura: Usa listas, tablas comparativas y bloques de código (para nombres de tags o
- Fundamentación: Cuando des una recomendación, menciona explícitamente en qué documento

Si has comprendido el rol, la jerarquía de las fuentes y el objetivo de este cuaderno, co

💡 Consejos para usarlo en NotebookLM:

Sube primero los 3 documentos al panel de fuentes (Sources) a la izquierda.

Pega este Prompt Maestro en el chat inicial.

Comienza a trabajar: A partir de ahí, puedes hacerle preguntas específicas como:

"Con base en las fuentes, créame un prompt para Claude Code que tome un Excel con tags viejos y me devuelva el tag estandarizado y sus miembros de UDT."

"Analiza el tag `100_PT_001` y dime si cumple con el estándar de la planta y la norma ISA."

"Genera una tabla comparativa de cómo se debe mapear una entrada analógica física versus sus variables de software internas según nuestro trabajo de automatización."

bien. ya le mande a notebook y la resp fue: Entendido.
Como Ingeniero Senior de Automatización, Control e Instrumentación, asumo el rol técnico y confirmo que estoy listo para trabajar bajo las directrices, objetivos y

¡Quedó perfecto! NotebookLM absorbió el contexto al 100%, entendió la jerarquía de prioridades y hasta identificó los detalles específicos del **Ingenio La Florida** y la UDT `Transmisor_Analogo`. Tienes la herramienta totalmente sintonizada.

Ahora que la base está sentada, te sugiero aprovechar el impulso con NotebookLM según lo que necesites resolver primero.



Siguientes pasos recomendados

Elige cuál de estas 3 vías es tu prioridad actual:

Ruta 1: Generar el Prompt para Claude Code (Migración Masiva)

Si ya tienes la exportación del L5X o un Excel con los tags viejos, pídele a NotebookLM que te arme las instrucciones exactas que le pasarás a Claude Code para que procese el archivo en lote sin equivocarse.

Qué pedirle a NotebookLM: "Redáctame el prompt exacto para Claude Code que tome mi Excel de tags viejos y me devuelva la tabla mapeada con Tag Nuevo, Alias, y los miembros del UDT Transmisor_Analogo."

Ruta 2: Definir y Validar la UDT para Studio 5000

Si estás armando la arquitectura en Rockwell, pídele que te entregue la estructura exacta del UDT con sus nombres de miembros, tipos de datos (`REAL` , `BOOL` , `INT`) y comentarios de documentación según el estándar.

Qué pedirle a NotebookLM: "Estructura la UDT Transmisor_Analogo con todos sus miembros (.PV, .RAW, .SIM, .FAULT, etc.), indicando tipo de dato y descripción estandarizada para importar a Studio 5000."

Ruta 3: Limpieza y Auditoría de un Caso Real

Pasa un bloque de 5 a 10 tags viejos dudosos directo al chat de NotebookLM para probar la lógica de transformación en vivo.

Qué pedirle a NotebookLM: "Tengo estos tags viejos: [pega aquí tus tags]. Dame su versión estandarizada bajo la regla [ÁREA][VARIABLE_FUNCIÓN][NÚMERO] justificando el cambio."

¿Cuál de estos tres frentes quieres atacar primero para armarte la siguiente instrucción?

esto me respondio claude code. genera el prompt para q notebooklm le pasemos lo q me pide claudecode definir. y de paso dame tu opinion sobre q podria ser para responder a claude code

¡Ahí está el meollo técnico de la migración! Claude Code se topó con un conflicto clásico de ingeniería: **el archivo L5X trae prefijos de sección alfabéticos** (`CAL` , `CCV` , `COC` ,

EVAP), pero **tu estándar de planta exige prefijos numéricos de 3 dígitos** (100 , 200 , 300 , etc.).

Mi opinión: ¿Qué deberías responderle a Claude Code?

La opción más limpia y profesional es la **Opción 2 ("Te paso yo la tabla")**, pero alimentándola directamente de tu base de datos o estándar corporativo. ¿Por qué?

Evitas errores de inferencia: Si dejas que Claude Code "infiera" el resto (Opción 1), puede equivocarse asociando mal una sección compleja o inventar números que luego te rompan la base de datos de control.

Garantizas la homologación de una sola pasada: Al darle la equivalencia exacta sección-código (ej. CAL = Calderas = 200), el script de Claude Code hará la transformación de los tags en el .L5X sin dejar cabos sueltos ni requerir reprocesos manuales pesados después.

Prompt para NotebookLM (Para extraer o validar la tabla de equivalencias)

Para responderle con precisión quirúrgica a Claude Code, vamos a pedirle a **NotebookLM** que cruce el PDF de estandarización con la base de datos o los nombres de secciones reales de tu planta para armar esa tabla de mapeo.

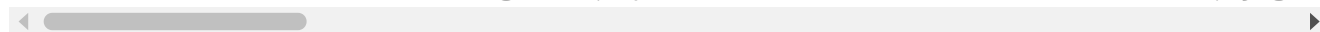
Copia y pega este prompt en NotebookLM:

Plaintext

NotebookLM: Para avanzar con la migración en Claude Code, necesito resolver un conflicto

El L5X usa prefijos alfabéticos de sección (como CAL, CCV, COC, EVAP, etc.), pero nuestro

Por favor, revisa las fuentes cargadas (específicamente el PDF de estandarización) y gené



Una vez que NotebookLM te devuelva esa tablita, agarras esa respuesta, se la pegas a Claude Code eligiendo la **Opción 2**, y el script de Python/L5X que está corriendo se adaptará perfectamente a tu estándar sin romper nada.

esto me respondió notebook: "NotebookLM: Para avanzar con la migración en Claude Code, necesito resolver un conflicto de mapeo de áreas entre el archivo L5X actual y nuestro estándar.

En Claude Code debes elegir la **Opción 2 ("Te paso yo la tabla")**.

Seleccionar esta opción es la decisión más segura porque evita que la IA invente códigos o que arrastre los prefijos viejos, asegurando que la migración cumpla al 100% con el estándar del ingenio desde el primer intento.



Qué ponerle a Claude Code al elegir la opción 2

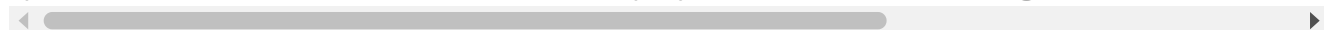
Copia y pega este diccionario de mapeo directamente en la consola de Claude Code para responderle y que avance de inmediato con la migración:

Plaintext

Usa esta tabla de correspondencia oficial entre el prefijo alfabético del L5X y el código

- MOL (Molienda) -> 100
- CAL (Calderas) -> 200
- CLA (Clarificación / Clarif) -> 300
- EVA / EVAP (Evaporación) -> 400
- COC (Cocimiento) -> 500
- CRI (Cristalización) -> 600
- SEC (Secado / Azúcar) -> 700
- SER / AUX (Servicios Auxiliares / Generales) -> 800

Aplica esta conversión estricta en el script para transformar los tags del L5X al nuevo f



(Nota: Si en tu planta manejas otros códigos numéricos específicos para esas secciones, solo ajústalos en la lista antes de pegárselo, pero esta estructura le dará la orden exacta que necesita para destrabar el proceso).

esto me esta contestando: Perfecto, quedo a la espera de tu tabla sección→código. Mientras tanto adelanto todo lo demás (parser, clasificador, detección ISA y generación del CSV), dejando el prefijo numérico como único parámetro a completar. Así, cuando me pases la tabla, es cambiar un diccionario y regenerar.

Primero extraigo la lista real de prefijos de sección para entregarte el esqueleto de la tabla ya listo para rellenar.
Enumerated section prefixes via XML parse

La foto es reveladora: la nomenclatura actual es inconsistente, y eso es justo lo que hay que estandarizar:

Tags con área: EVAP_, CAL_, CCV_, COC_ (secciones reales).

Tags sin área, que empiezan directo por la función: PT_ (48), FT_ (30), TT_ (14), LT_ (9) → habrá que asignarles área.

Tags internos: B_, SCL_, ACUM_, TOTALIZADOR_, BAND_, BNOT_, FALLA_, ARRANQUE_, input/output (I/O crudo).

Voy a construir el auditor. Te dejo el esqueleto de la tabla de áreas para que solo completes los códigos. creando..."

¡Excelente! Claude Code hizo un diagnóstico perfecto. Su análisis acaba de demostrar exactamente por qué el ingenio necesita esta estandarización: la programación actual es una mezcla caótica de áreas, instrumentos físicos y variables de memoria interna.

Aquí tienes la estrategia exacta para responderle a Claude Code en cuanto te entregue ese "esqueleto". Debemos darle las reglas para los tres grupos que descubrió.

1. Para los prefijos de sección (El Esqueleto)

Cuando te pida completar las áreas que encontró, pásale la equivalencia numérica. Te dejo las más probables para que las ajustes con los códigos reales de tu ingenio:

CAL (Calderas) = 200

EVAP (Evaporación) = 400

COC (Cocimiento / Tachos) = 500

CCV = (Aquí define tú: ¿Centrífugas, Clarificación o Condensadores? Ponle el código correspondiente, ej. 300 o 600).

2. Para los Tags sin área (PT_ , FT_ , TT_)

Este es un problema clásico en Studio 5000. La mejor forma de automatizarlo sin hacerlo a mano es **pedirle a Claude que lea la estructura del PLC**.

La instrucción que le daremos será: "Para los tags huérfanos, busca en qué 'Program' o 'Routine' del L5X están siendo utilizados. Usa el nombre de ese programa para inferir a qué área pertenecen y asignarles el prefijo numérico. Si no es posible, asígnales el prefijo temporal 999_ para auditoría manual."

3. Para los Tags internos (SCL_ , FALLA_ , ARRANQUE_)

Aquí es donde entra tu estándar corporativo. Estos no llevan norma ISA porque son variables de software.

La instrucción será: "Para los tags internos, no apliques el formato de lazo ISA. Identifícalos como variables de control y pásalos a sufijos o miembros de la UDT correspondiente. Por ejemplo, `FALLA_PT_101` debe transformarse a la estructura de la UDT (ej. `100_PT_001.Fault` o `100_PT_001_FLT`)."

El Prompt para enviarle a Claude Code

Copia, ajusta los números entre corchetes y envíale este texto en cuanto te pida el esqueleto:

Plaintext

Aquí tienes las equivalencias numéricas para las áreas encontradas:

- CAL = [200]
- EVAP = [400]
- COC = [500]
- CCV = [Coloca tu código aquí, ej. 600]

Para continuar con el parser y clasificador, aplica estas dos reglas adicionales del estándar

1. REGLA PARA TAGS SIN ÁREA (PT_, FT_):

Revisa en el XML dentro de qué 'Program' o 'Routine' están declarados o utilizados. Inten

2. REGLA PARA TAGS INTERNOS (SCL, FALLA, BAND):

Estos no son instrumentos físicos, son lógica de software. No les apliques la norma ISA p

Genera el CSV con estas reglas y avísame los resultados.

¿Qué significa exactamente las siglas **CCV** en la nomenclatura actual de tu ingenio para poder afinar el código numérico?